

Application Device Queues (ADQ) For System-level Network I/O Performance Improvements

Aug 2020, Netdev 0x14

Kiran Patil
Amritha Nambiar
Sridhar Samudrala

Agenda

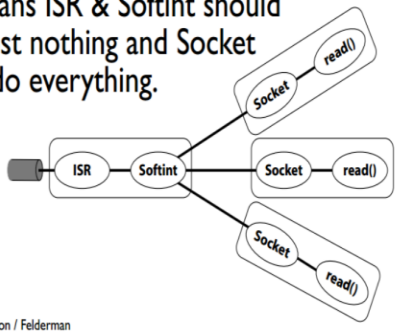
- Motivation: Speeding up Networking
- System Inefficiencies
- What is ADQ ?
- ADQ Ingredients
- ADQ putting it all together
- Performance
- Benefits of ADQ
- Summary/Addressing system inefficiencies
- What is next?
- References

Motivation: Speeding up Networking

The end of the wire isn't the end of the net

On a uni-processor it doesn't matter but on a multi-processor the protocol work should be done on the processor that's going to consume the data.

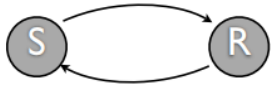
This means ISR & Softint should do almost nothing and Socket should do everything.



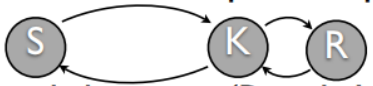
LCA06 - Jan 27, 2006 - Jacobson / Felderman

Internet Stability

- You can view a network connection as a servo-loop:



- A kernel-based protocol implementation converts this to two coupled loops:



- A very general theorem (Routh-Hurwitz) says that the two coupled loops will always be less stable then one.

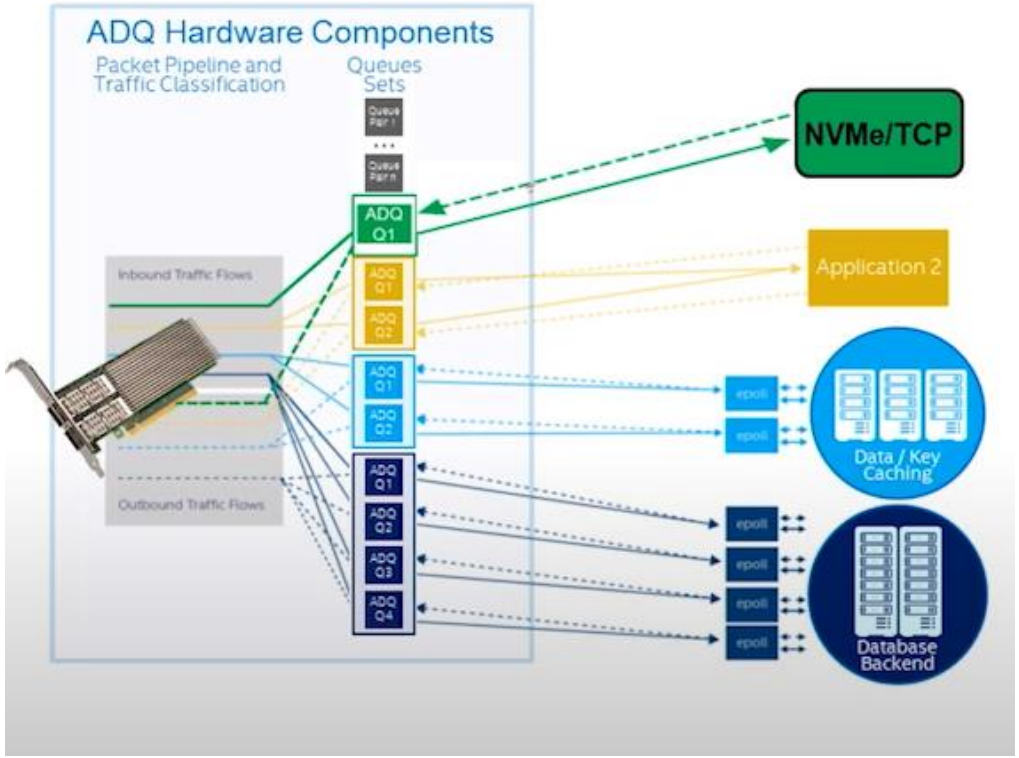
The kernel loop hides the receiving app dynamics from sender which complicates RTT estimates & causes spurious retransmission.

System Inefficiencies : Influenced by NIC/stack/app

System attribute	Problem(s) to influence/solve
Interrupts	How to prevent an interrupt from firing when an application is doing useful work?
Context Switches	How to minimize context switches?
Synchronization	How to minimize/eliminate synchronization operations e.g. locks, multi-thread sharing?
Application & Protocol Processing	How can application and protocol processing operate in the same context?
Data Movement	How to minimize data movement during packet processing for control and data?

The last mile – “End of the wire is not the End of the Net”, *Van Jacobson*

Application Device Queues (ADQ)



What is ADQ ?

- It is not a protocol
- It is application specific queuing and steering technology

ADQ Basics

- Filters application traffic to a dedicated set of queues
- Application threads of execution are connected to specific queues within the ADQ queue set
- Bandwidth control of egress (Tx) network traffic per application

Source: SDC presentation
https://www.youtube.com/watch?v=JNgx0jBiYmM&list=PLH_ag5Km-YUY9kkdOYujfrdQjEIh83aNF&index=32&t=0s

ADQ Ingredients

1. Configure application priority

- cgroup net_prio, cgroup BPF, socket option (SO_PRIORITY)

2. Queue-set configuration (tc-mqprio)

- HW offload for queue-set (per TC queue-set configuration)
- Traffic shaping (HW offload for egress rate limiting (min and max))
- Leverage existing mechanism of mapping prio->TC
- iproute2:tc extension

```
# tc qdisc add dev eth0 root mqprio num_tc 4 map 0 1 2 3 \
  queues 4@0 4@4 4@8 4@12 hw 1 mode channel \
  shaper bw_rlimit min_rate 1Gbit 2Gbit 2Gbit 1Gbit \
  max_rate 4Gbit 5Gbit 5Gbit 10Gbit
```

ADQ Ingredients cont.

3. Queue-set isolation (tc-flower)

- HW offload for traffic filtering per queue-set
- iproute2:tc extension

4. Queue selection

- Queue isolation within a queue-set using RSS/Flow director rule in HW

```
# ethtool -K eth0 hw-tc-offload on
# tc qdisc add dev eth0 ingress

# tc filter add dev eth0 protocol ip ingress prio 1 flower\
  dst_ip 192.168.0.1/32 ip_proto tcp \
  dst_port 11211 skip_sw hw_tc 1

# tc filter add dev eth0 protocol ip ingress prio 1 flower\
  dst_ip 192.168.0.1/32 ip_proto tcp \
  dst_port 5000 skip_sw hw_tc 2

# tc filter add dev eth0 protocol ip ingress prio 1 flower\
  dst_ip 192.168.0.1/32 ip_proto tcp \
  dst_port 6000 skip_sw hw_tc 3
```

ADQ Ingredients cont.

5. Busy polling

- recv ( ), poll ( )
- epoll ( ): busy_poll

```
sysctl net.core.busy_poll=50
```

6. Queue Identification (SO_INCOMING_NAPI_ID)

- NAPI-ID associated with queue vector identifies queue
- Alignment of application thread to device queue
- Single producer-consumer data flow, avoiding synchronizations/contentions

ADQ Ingredients cont.

7. Extension to XPS (Transmit Packet Steering)

- XPS is normally CPU based.
- Rx queue(s) based transmit packet steering
- Facilitate proactive alignment of transmit and receive queue
- Facilitate single producer and consumer model effectively
- User driven optional configuration (sysfs)
- Write ‘queue_mask’ to Tx queue for xps_rxqs attribute

```
echo 1 > /sys/class/net/eth0/queues/tx-0/xps_rxqs

echo 2 > /sys/class/net/eth0/queues/tx-1/xps_rxqs
```

ADQ Putting It All Together

Linux Kernel Enhancements	Version
Event polling (epoll enabled with busy_polling device queue)	4.12
Queue Identification for application (Socket option for NAPI-ID)	4.12
TC-Mqprio hardware offload and shaper	4.15
Switch filter with TC flower	4.15
XPS based on receive queue	4.19

Align Application Thread To Queue



Open source memcached patch:
https://github.com/memcached/memcached/commit/0899abab54a85ca8b06eb2e294d3bc2888285626

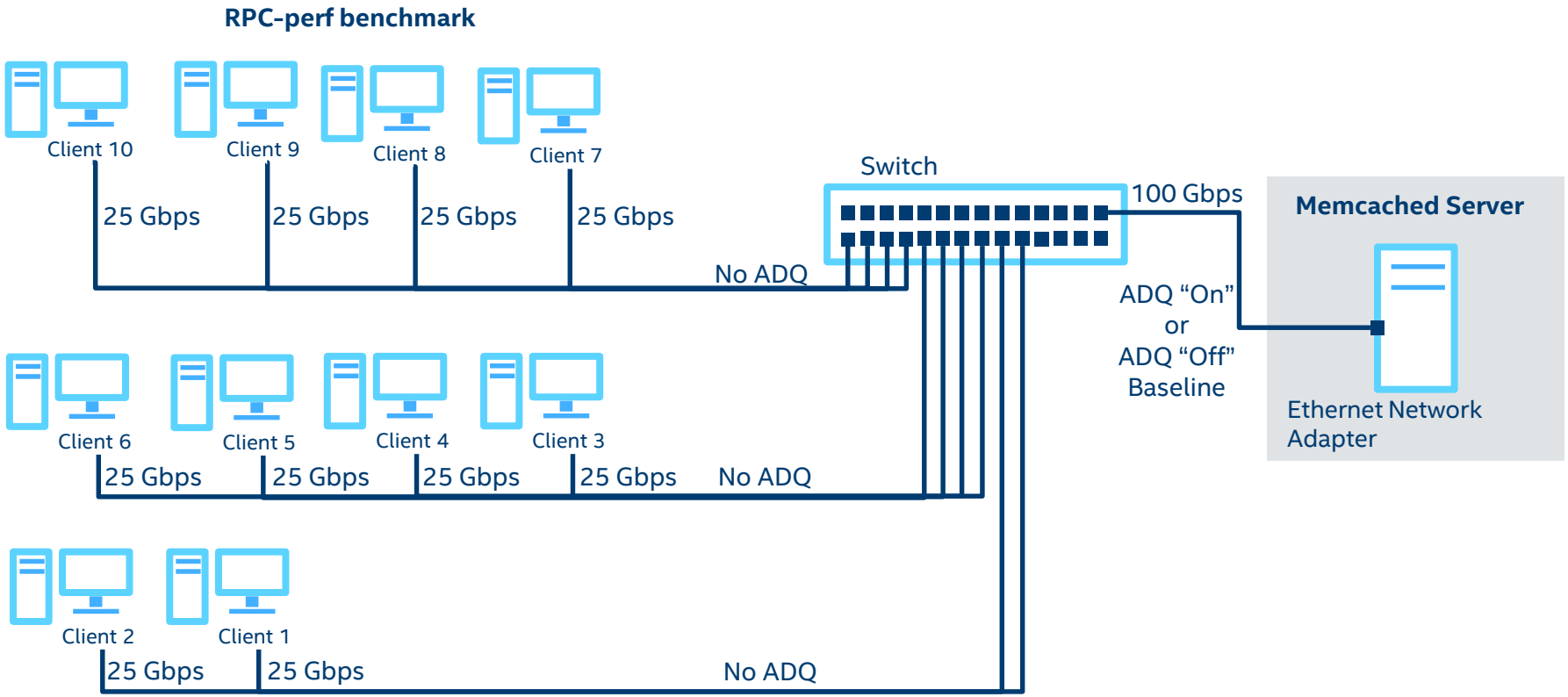
```
diff --git a/thread.c b/thread.c
index b02b0f9..1f2600b 100644
--- a/thread.c
+++ b/thread.c
@@ -452,6 +452,20 @@ void dispatch_conn_new(int sfd, enum conn_states init_state, int event_flags,

    LIBEVENT_THREAD *thread = threads + tid;

+   {
+       int napi_id;
+       socklen_t len = sizeof(napi_id);
+
+       if (getsockopt(sfd, SOL_SOCKET, SO_INCOMING_NAPI_ID, &napi_id, &len) == -1) {
+           perror("getsockopt SO_INCOMING_NAPI_ID");
+       } else {
+           if (napi_id != -1)
+             thread = threads + (napi_id % settings.num_threads);
+       }
+   }
```

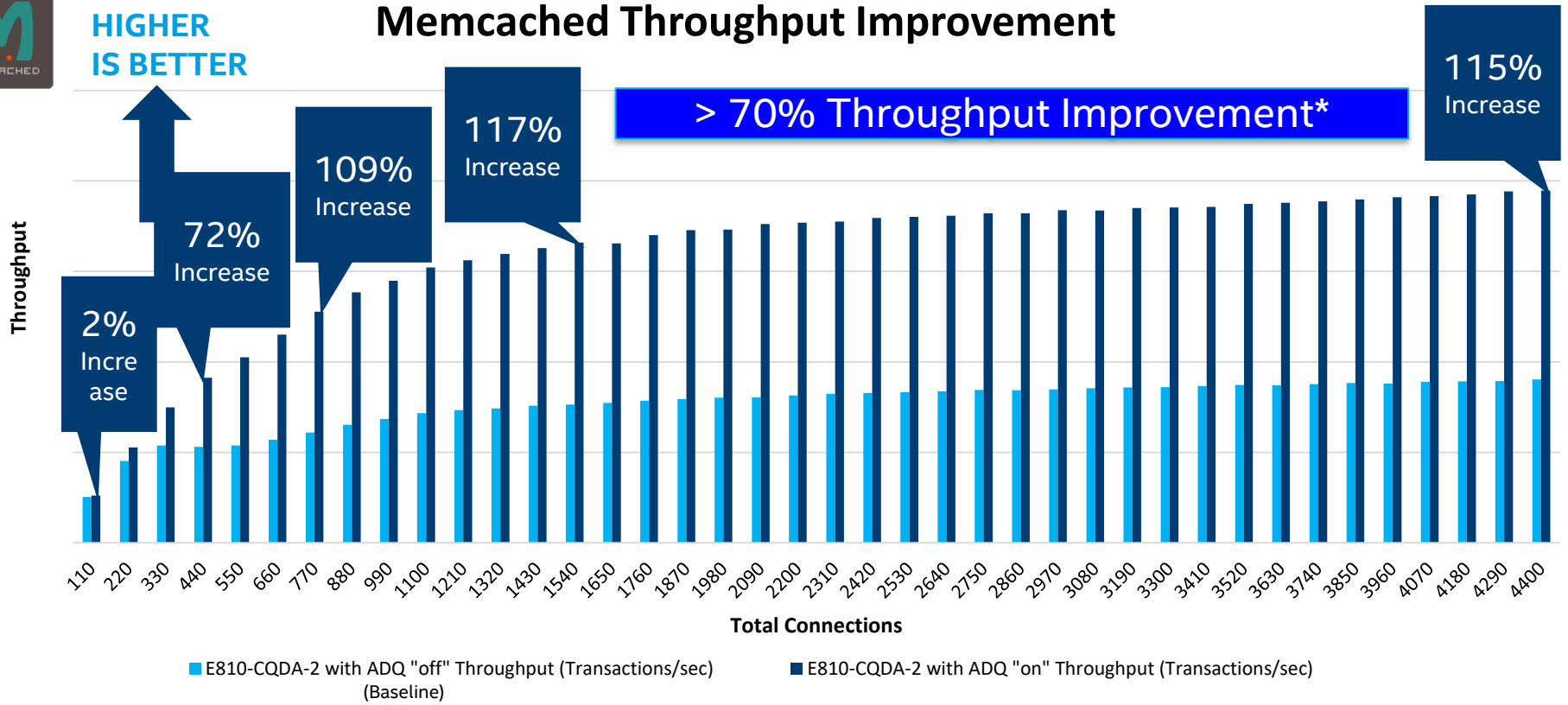
Performance

Test Topology



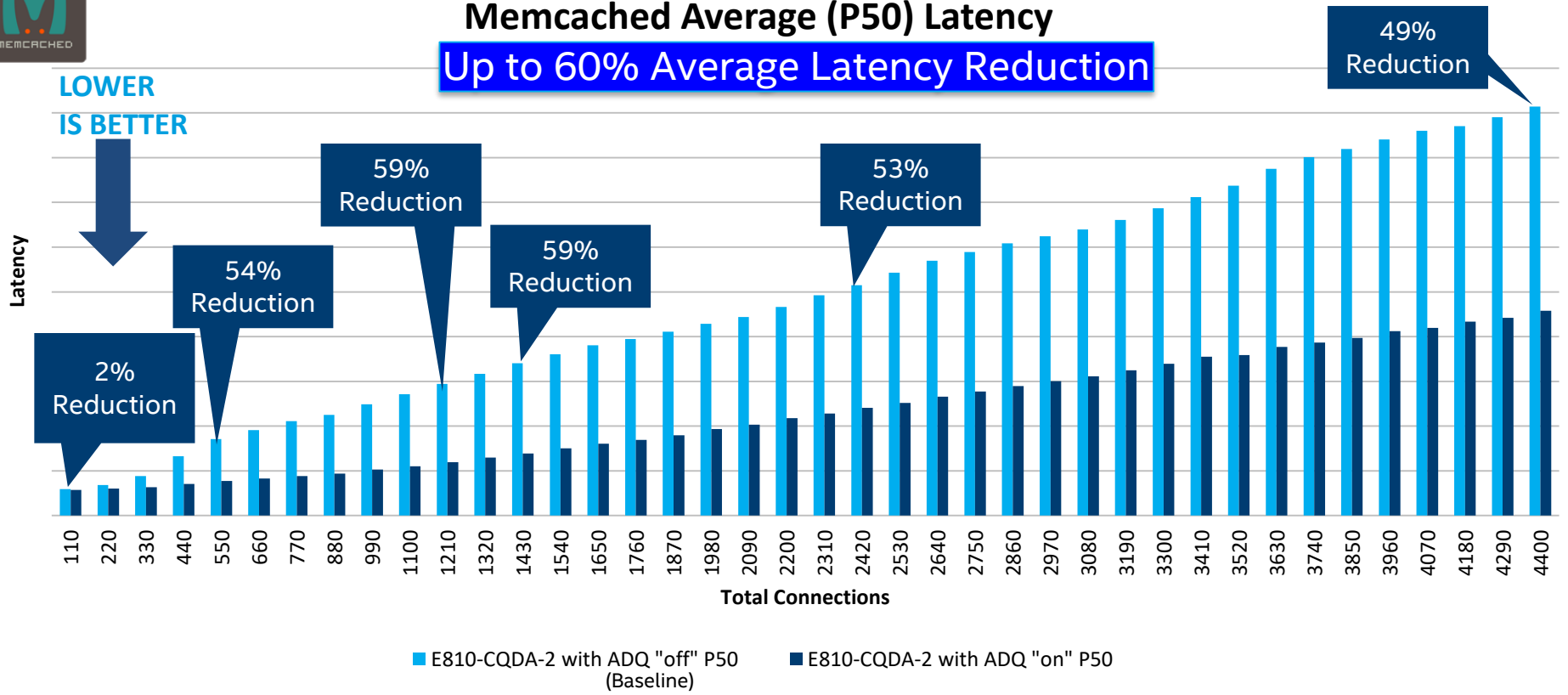
Memcached Throughput Improvement

HIGHER IS BETTER

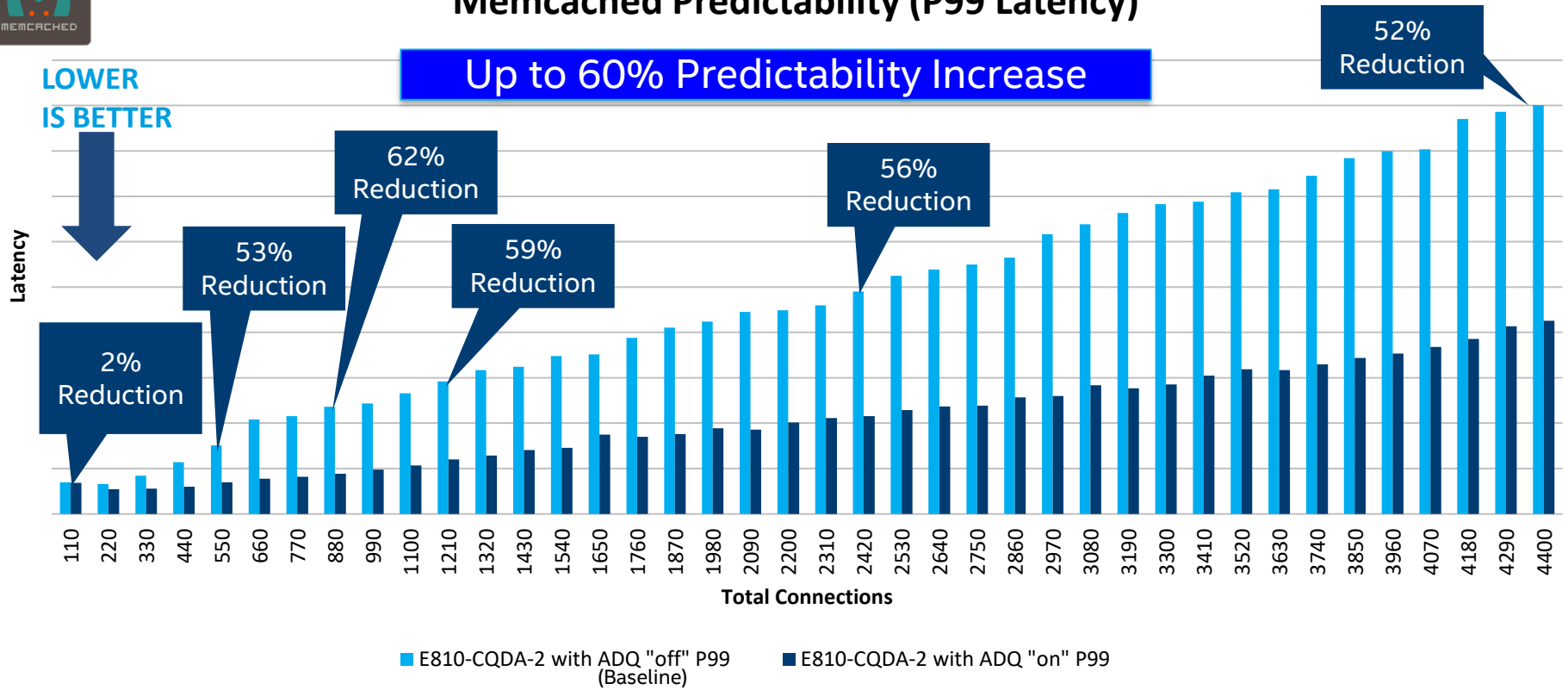


Memcached Average (P50) Latency

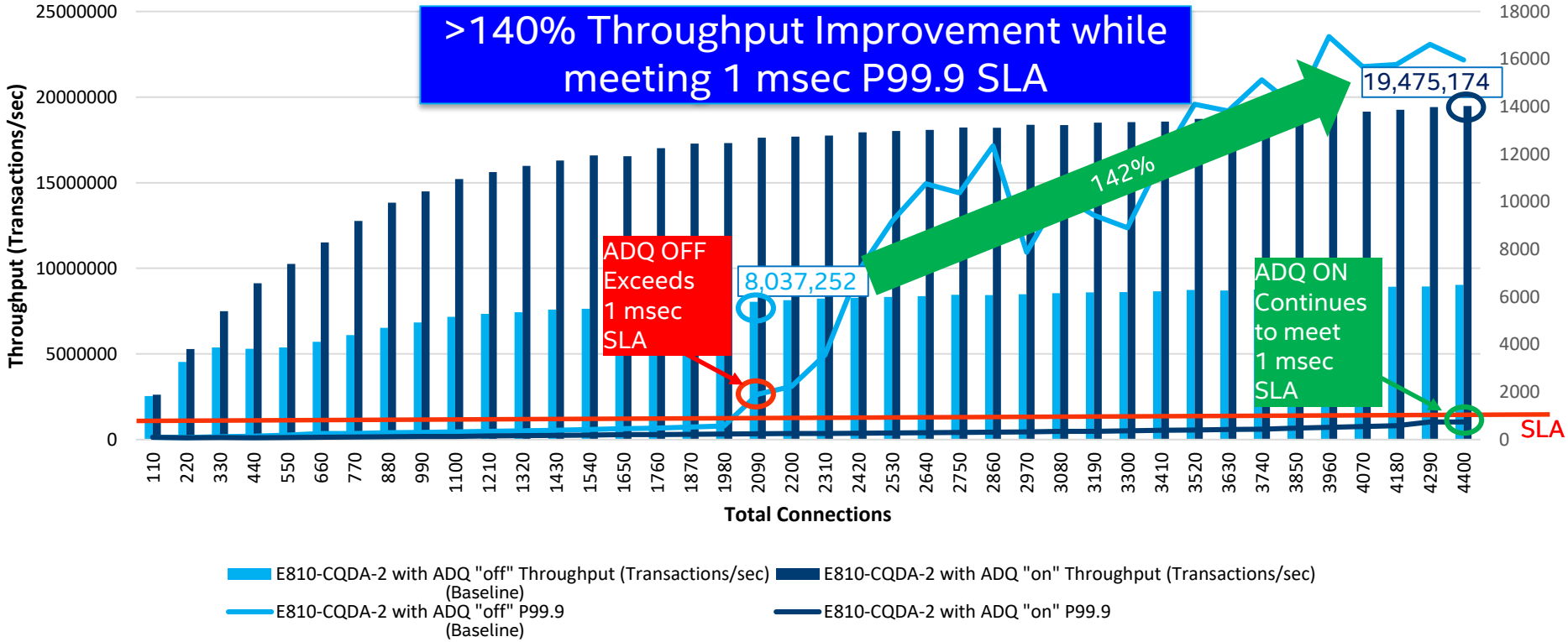
Up to 60% Average Latency Reduction



Memcached Predictability (P99 Latency)



Memcached Throughput Comparison while meeting 1 msec SLA for P99.9 Latency



For more complete information about performance and benchmark results, visit www.intel.com/benchmarks. See configuration disclosure for details. For more information regarding performance and optimization choices in Intel software products., please visit https://software.intel.com/en-us/articles/optimization-notice.

Benefits of ADQ

INCREASES APPLICATION **PREDICTABILITY**



REDUCES APPLICATION **LATENCY**



IMPROVES APPLICATION **THROUGHPUT**



Summary/Addressing system inefficiencies

System attribute	ADQ improvements
Interrupts	Reduce number of interrupts significantly during application activity, assisted by event based busy polling
Context Switches	Event based busy polling keeps application context alive during application related communications
Synchronization	Single producer consumer model creates a unique pipe between an application thread and a device queue
Application & Protocol Processing	Protocol processing triggers from the application. Transmit rate limiting per application.
Data Movement	Single producer consumer data flows and event based busy polling constrain data movement boundaries

What Is Next?

- Linux max limit on number of TCs (16)

```
include/linux/netdevice.h

#define TC_MAX_QUEUE    16
```

  - Limits acceleration to 15 applications
- Lack of additional configurability per TC (such as per queue-set interrupt tuning)
- A single interface to configure both Tx and Rx queue-sets
  - Tx queue-set configuration using tc-mqprio
  - TC does not support Rx queue-set configuration

References

- Linux Conf 2006: Speeding up Networking :“End of the wire is not the end of the net” http://www.lemis.com/grog/Documentation/vj/lca06vj.pdf
- Epoll busy poll https://lwn.net/Articles/551284/
- Epoll busy poll works with SO_RESUSE_PORT https://lwn.net/Articles/542629/
- “XPS using receive queues” in the following Linux kernel document for symmetric queue https://www.kernel.org/doc/Documentation/networking/scaling.txt
- SDC Presentation https://www.youtube.com/watch?v=JNgx0jBiYmM&list=PLH_ag5Km-YUY9kkdOYujfrdQjEIh83aNF&index=32&t=0s
- Latest version of the ADQ Configuration Guide and Demo Code Package is available at https://wiki.ith.intel.com/pages/viewpage.action?pageId=1334587222


intel
experience
what's inside™